



Vulnerability Assessment & Penetration Testing Report

Target:

<https://bank.offenso.local/#/>

Prepared For:

Hackware Security Solutions LLP

Prepared By:

Basith Mohammed Ali

Date:

09 July 2026

Version:

1.0

Table of Contents

1. Executive Summary.....	3
2. Scope.....	3
3. Methodology.....	3
4. Findings Summary.....	4
F-01 – Network Service Enumeration.....	5
F-02 – Authentication Session Hijacking via Missing Secure Cookie Attribute.....	6
F-03 – Stored Cross-Site Scripting (XSS) via User Profile Name.....	9
F-04 – Insufficient OTP Brute-Force Protection.....	11
F-05 – Insufficient Brute-Force Protection on Two-Factor Authentication (2FA) OTP Verification.....	12
F-06 – Clickjacking Due to Missing Frame Protection.....	13
F-07 – Sensitive Data Exposure Through Full Payment Card Number Disclosure.....	16
F-08 – Missing Rate Limiting on Authentication Endpoint.....	17
F-09 – Business Logic Flaw Allowing Invalid Transfer Amounts.....	18
Conclusion.....	21

1. Executive Summary

A Vulnerability Assessment and Penetration Testing (VAPT) engagement was conducted on the **Offenso Bank** web application (<https://bank.offenso.local>) to identify security weaknesses that could impact the confidentiality, integrity, and availability of the application. Multiple vulnerabilities were identified across authentication, session management, input validation, business logic, and client-side security. The findings demonstrate the potential for unauthorized account access, sensitive information disclosure, client-side attacks, and abuse of application functionality. Detailed technical findings and remediation recommendations are provided in this report.

2. Scope

The assessment was performed against the Offenso Bank web application hosted at:

Target	https://bank.offenso.local
--------	---

The testing focused on application functionality, authentication mechanisms, authorization controls, session management, input validation, business logic, and client-side security.

3. Methodology

Phase	Description
Reconnaissance & Enumeration	Identification of exposed services, application functionality, and accessible endpoints.
Vulnerability Identification	Discovery of security weaknesses through manual testing and automated tools.
Vulnerability Validation	Manual verification of identified vulnerabilities to eliminate false positives.
Risk Assessment	Evaluation of the business impact and severity of confirmed findings.
Reporting	Document findings, impact, and remediation.

4. Findings Summary

ID	Finding	Severity
F-01	Network Service Enumeration	Informational
F-02	Authentication Session Hijacking via Missing Secure Cookie Attribute	High
F-03	Stored Cross-Site Scripting (XSS) via User Profile Name	Medium
F-04	Insufficient OTP Brute-Force Protection	Medium
F-05	Insufficient Brute-Force Protection on Two-Factor Authentication (2FA) OTP Verification	Medium
F-06	Clickjacking Due to Missing Frame Protection	Medium
F-07	Sensitive Data Exposure Through Full Payment Card Number Disclosure	Medium
F-08	Missing Rate Limiting on Authentication Endpoint	Medium
F-09	Business Logic Flaw Allowing Invalid Transfer Amounts	Medium

Severity Summary

Severity	Count
High	1
Medium	7
Low	0
Informational	1

F-01 – Network Service Enumeration

Severity	Informational
CVSS v3.1	N/A
Target	192.168.56.112

Description

Network reconnaissance identified multiple publicly accessible services exposed by the target host. The discovered services include SSH, HTTP, HTTPS, and a Node.js Express application. This information defines the application's external attack surface and serves as the foundation for subsequent security testing.

Proof of Concept

Step 1- Perform an Nmap service and version detection scan.

```
nmap -A -T4 192.168.56.112
```

Step 2- Review the identified services

Port	Service	Version
22/TCP	SSH	OpenSSH 9.9p1
80/TCP	HTTP	nginx 1.26.3
443/TCP	HTTPS	nginx 1.26.3
3001/TCP	HTTP	Node.js Express

The scan also identified the target operating system as Linux and confirmed the presence of a valid TLS certificate for the `*.offenso.local` domain.

Impact

The identified services define the application's attack surface and may assist attackers in planning further security assessments or targeted attacks.

Recommendation

Regularly review exposed services, disable unnecessary ports, restrict administrative access, and ensure all services are kept up to date with the latest security patches.

F-02 – Authentication Session Hijacking via Missing Secure Cookie Attribute

Severity	High
CVSS v3.1	8.1
Affected Component	HTTP Access (http://bank.offenso.local/) Authentication Session Cookie (token)

Description

The application allows authenticated users to access the application over HTTP before redirecting them to HTTPS. The authentication session cookie (**token**) is configured **without the Secure attribute**, causing the browser to automatically transmit the cookie over an unencrypted HTTP connection. This allows an attacker with network access to capture the session cookie and reuse it to impersonate the authenticated user, resulting in session hijacking without requiring valid credentials or multi-factor authentication.

Proof of Concept

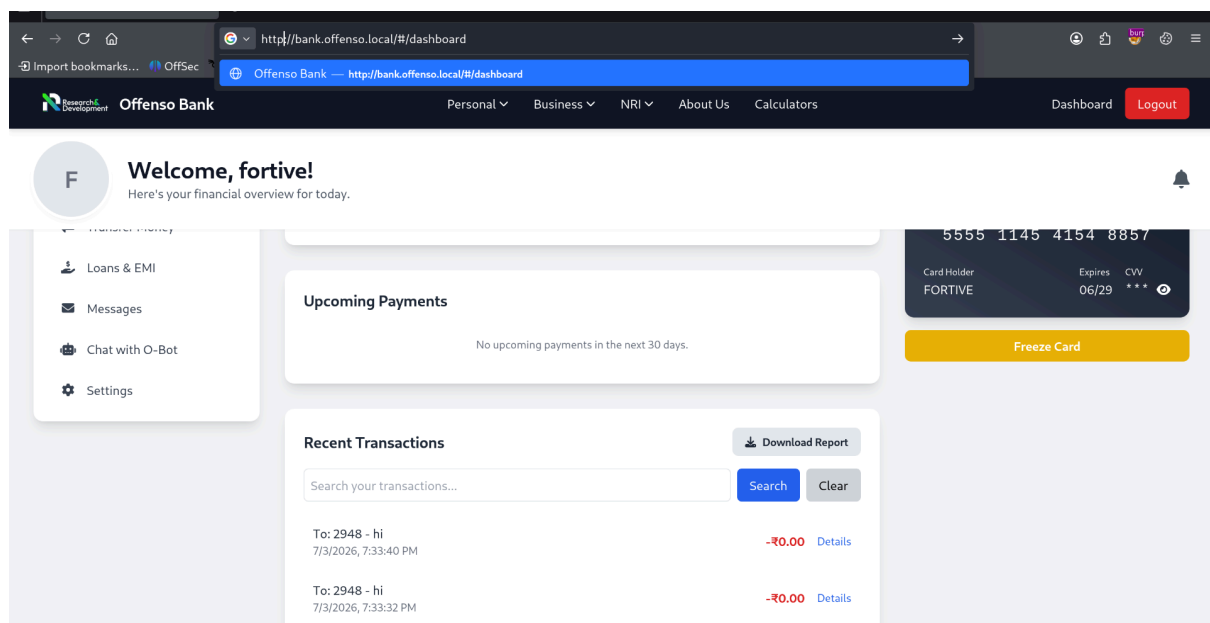
Step 1 – Authenticate to the application

Login as **User A** using valid credentials.

Step 2 – Capture the session cookie

While the authenticated session is active, access the application using:

`http://bank.offenso.local`



Before the application redirects the request to HTTPS, the browser automatically sends the authentication cookie over HTTP. The request was captured using Wireshark, revealing the session token in cleartext.



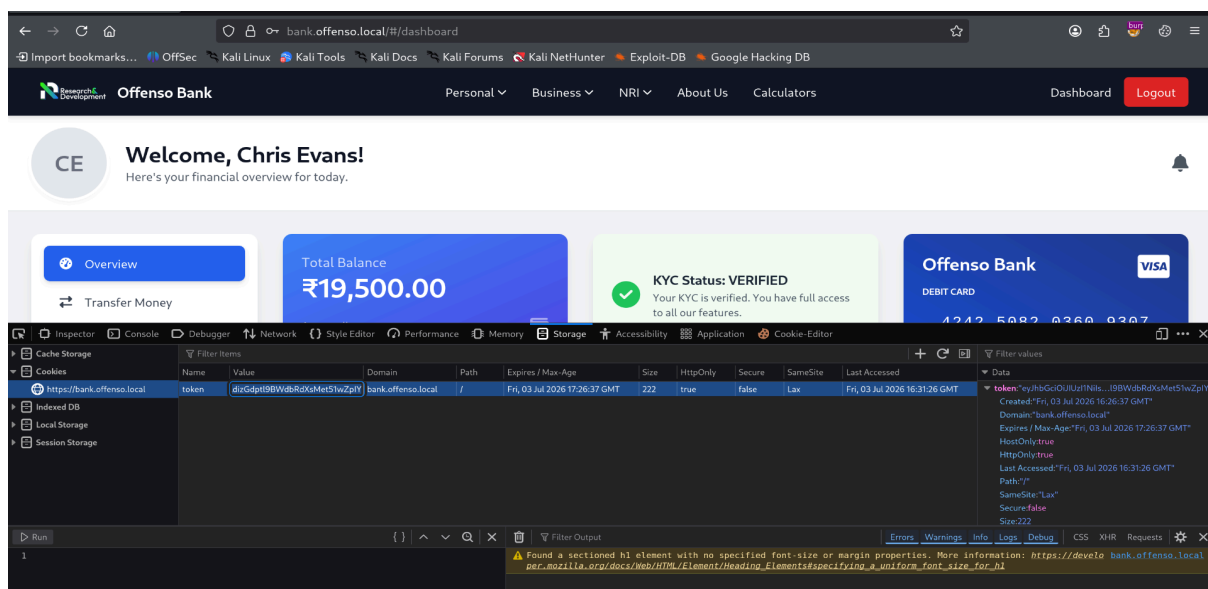
```
GET / HTTP/1.1
Host: bank.offenso.local
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjZhbm90Y2U3ZGRkNTU5N2IxNjU0ZGZiZiIsIm5hbWUiOiJmb3J0aXZlIiwicm9sZSI6InVzZXIiLCJpYXQiOiJlbnR5c3R5bW0TU4NjMsImV4cCI6MTc4MzA5OTQ2M30.Irj-sSdpsuwe3CLfmN8zgfmo11R2KjXtIhyfSF2IBxg
Upgrade-Insecure-Requests: 1
Priority: u=0, i

HTTP/1.1 301 Moved Permanently
Server: nginx/1.26.3 (Ubuntu)
Date: Fri, 03 Jul 2026 16:27:21 GMT
Content-Type: text/html
Content-Length: 178
Connection: keep-alive
Location: https://bank.offenso.local/

<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx/1.26.3 (Ubuntu)</center>
</body>
</html>
```

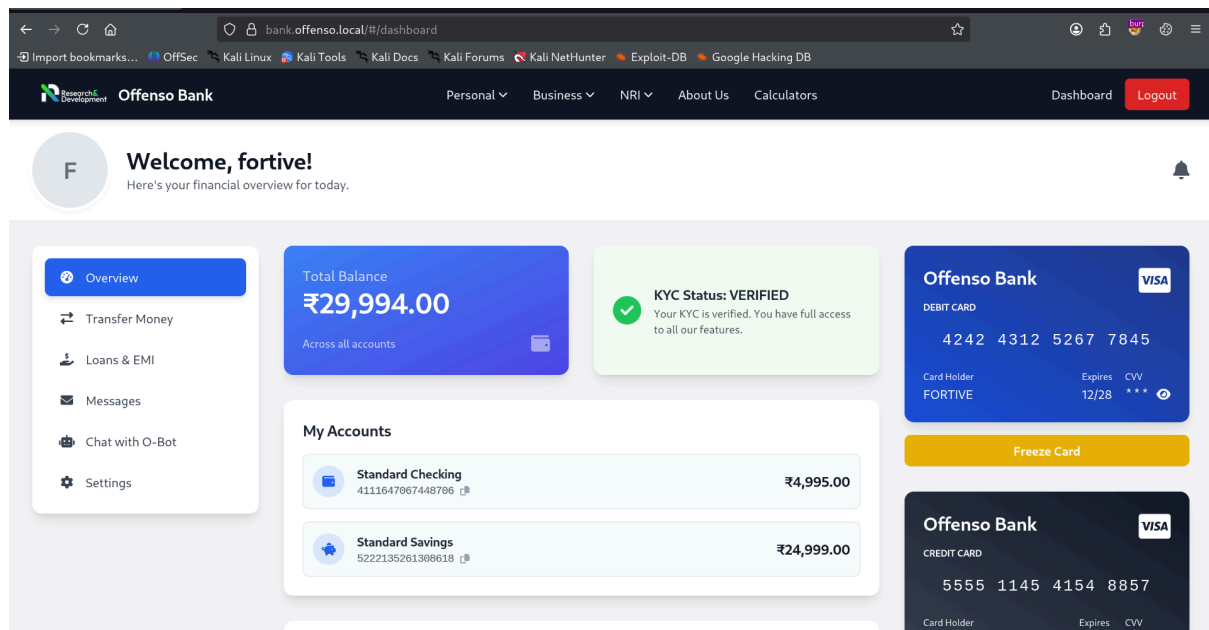
Step 3 – Reuse the captured cookie

Open a separate browser session and authenticate as **User B**. Replace User B's authentication cookie with the captured session cookie obtained from User A.



Step 4 – Verify session hijacking

Refresh the page after replacing the cookie. The application loads **User A's account**, demonstrating successful session hijacking.



Impact

An attacker with the ability to intercept network traffic can capture authenticated session cookies and reuse them to gain unauthorized access to another user's account. This may expose sensitive banking information and allow unauthorized actions to be performed using the victim's active session.

Recommendation

Configure the authentication cookie with the Secure attribute.

Enforce HTTPS across the entire application.

Enable HTTP Strict Transport Security (HSTS) to prevent HTTP access.

Reject authenticated requests received over HTTP.

Continue using the HttpOnly attribute and consider using SameSite=Strict where appropriate.

F-03 – Stored Cross-Site Scripting (XSS) via User Profile Name

Severity	Medium
CVSS v3.1	5.4
Affected Endpoint	PUT /api/user/profile

Description

The application does not properly sanitize or encode user-supplied input before storing and rendering it. An authenticated user can inject malicious JavaScript into the profile name, which is later executed when the application displays the stored value in another part of the application (O-Bot page). This results in a **Stored Cross-Site Scripting (Stored XSS)** vulnerability.

Proof of Concept

Step 1 – Update the Profile Name with a Malicious Payload

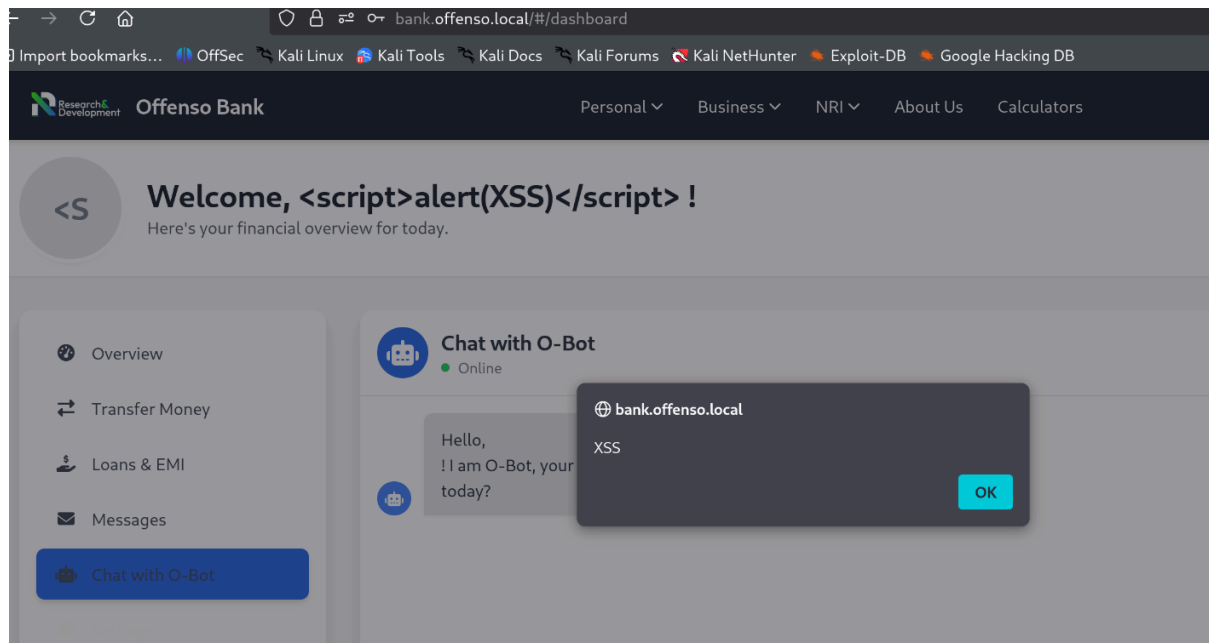
Navigate to **Settings** and update the profile name with the following payload:

```
<img src=x onerror=alert('XSS')>
```

Click **Save Changes**.

Step 2 – Trigger the Stored Payload

Navigate to the **Chat with O-Bot** page. The application loads the stored profile name and renders it without proper output encoding, causing the injected JavaScript to execute.



Impact

An attacker can store malicious JavaScript that executes when the affected profile information is rendered. Successful exploitation may allow session hijacking, sensitive information theft, phishing attacks, unauthorized actions on behalf of the victim, or further compromise of authenticated users depending on the execution context.

Recommendation

Validate and sanitize all user-supplied input before storing it.

Apply context-aware output encoding whenever user-controlled data is rendered.

Avoid rendering untrusted HTML.

Implement a strong Content Security Policy (CSP) to reduce the impact of XSS.

Perform server-side input validation in addition to client-side validation.

F-04 – Insufficient OTP Brute-Force Protection

Severity	Medium
CVSS v3.1	5.3
Affected Endpoint	POST /api/auth/reset-password

Description

The password reset functionality does not enforce sufficient protection against repeated OTP verification attempts. Multiple invalid OTP values can be submitted without triggering rate limiting, temporary account lockout, or request throttling, allowing attackers to repeatedly guess OTP values until the correct one is found.

Proof of Concept

Step 1: Capture the password reset request to `/api/auth/reset-password` and send it to Burp Suite Intruder.

Step 2: Configure the `otp` parameter as the payload position and launch an Intruder attack using multiple six-digit OTP values.

Step 3: The application processed over **100 consecutive OTP attempts** without blocking the requests or returning a rate-limit response.

Step 4: When the correct OTP was submitted (request **#101**), the server returned **HTTP/1.1 200 OK** with the message:

The screenshot displays the Burp Suite Intruder interface. The top panel shows a list of 100 requests, each with a status code of 400. The bottom panel shows the details of the 101st request, which was successful (HTTP/1.1 200 OK). The request headers include: Accept-Language: en-US,en;q=0.5, Accept-Encoding: gzip, deflate, br, Content-Type: application/json, Content-Length: 65, Origin: https://bank.offense.local/, Referer: https://bank.offense.local/, Sec-Fetch-Dest: empty, Sec-Fetch-Mode: cors, Sec-Fetch-Site: same-origin, Priority: uo, Te: trailers, and Connection: keep-alive. The request body is a JSON object: {"email": "red@offense.local", "otp": "238692", "newPassword": "test"}. The response headers include: Server: nginx/1.26.3 (Ubuntu), Date: Fri, 03 Jul 2026 06:03:23 GMT, Content-Type: application/json; charset=utf-8, Content-Length: 51, Connection: keep-alive, X-Powered-By: Express, Vary: Origin, Access-Control-Allow-Credentials: true, and ETag: W/"33:J7UmuxT9zR90x0YFEY5p04H4s". The response body is a JSON object: {"message": "Password has been reset successfully."}.

Impact

An attacker who knows a valid user's email address may repeatedly guess OTP values without being blocked. If the correct OTP is discovered, the attacker can reset the victim's password and gain unauthorized access to the account.

Recommendation

Implement rate limiting and temporary account lockout for repeated OTP failures. Additionally, limit the number of OTP verification attempts, enforce OTP expiration, and introduce increasing delays after consecutive failed attempts.

F-05 – Insufficient Brute-Force Protection on Two-Factor Authentication (2FA) OTP Verification

Severity	Medium
CVSS v3.1	5.3
Affected Endpoint	POST /api/auth/verify-otp

Description

The application enforces a **5-minute validity period** for 2FA OTPs; however, it does not implement sufficient rate limiting, request throttling, or temporary account lockout during this period. As a result, an attacker can repeatedly submit OTP values until the correct one is accepted before the OTP expires.

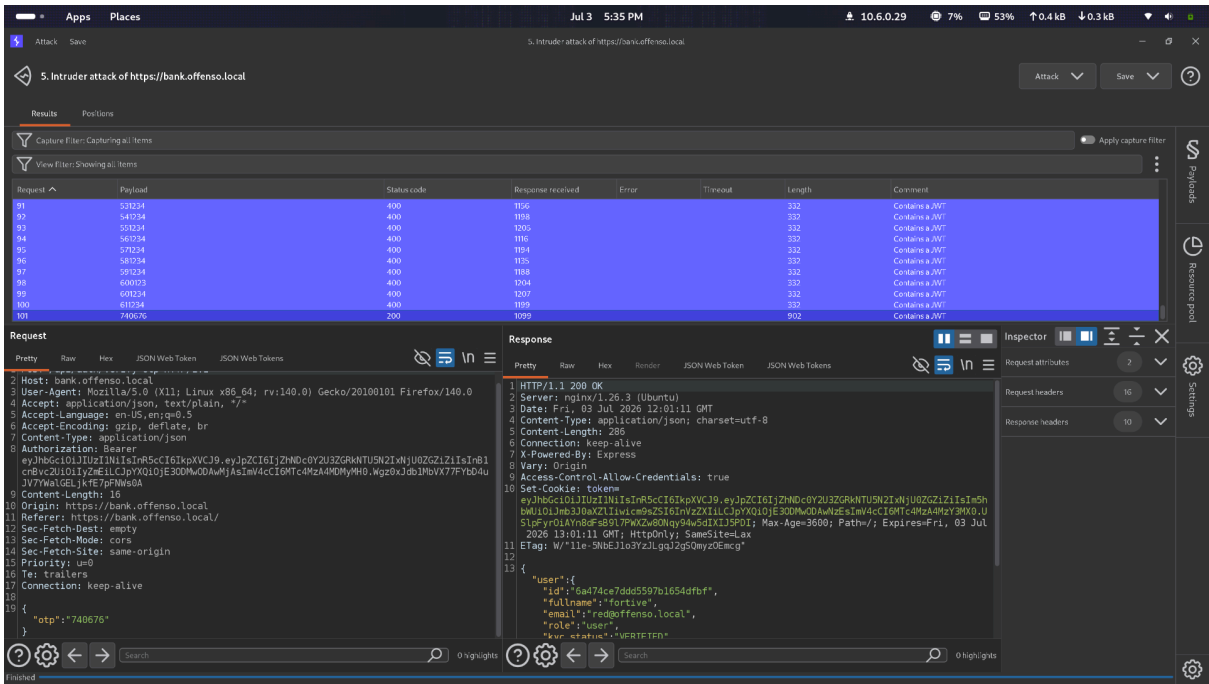
Proof of Concept

Step 1: Capture the `POST /api/auth/verify-otp` request and send it to Burp Suite Intruder.

Step 2: Configure the `otp` parameter as the payload position and launch an Intruder attack using multiple six-digit OTP values.

Step 3: The application accepted over 100 consecutive OTP verification requests without blocking or throttling the client.

Step 4: On request **#101**, the correct OTP was submitted and the server returned **HTTP/1.1 200 OK**, successfully completing the 2FA verification and issuing an authenticated session.



Impact

An attacker who has obtained valid login credentials may repeatedly guess OTP values during the OTP validity period. If the correct OTP is discovered, the attacker can successfully bypass the second authentication factor and gain unauthorized access to the victim's account.

Recommendation

Implement rate limiting and temporary account lockout after multiple failed OTP attempts. Additionally, limit the maximum number of OTP verification attempts, enforce OTP expiration, and introduce progressive delays between failed requests.

F-06 – Clickjacking Due to Missing Frame Protection

Severity	Medium
CVSS v3.1	6.5
Affected Endpoint	https://bank.offenso.local/#/login

Description

The application allows its login page to be embedded within an external website because it does not implement standard anti-clickjacking protections, such as the **X-Frame-Options** response header or the **Content-Security-Policy (frame-ancestors)** directive. As a result, an attacker can display the legitimate banking application inside a malicious webpage, overlay deceptive interface elements, and trick users into entering sensitive information or performing unintended actions while believing they are interacting with the genuine application.

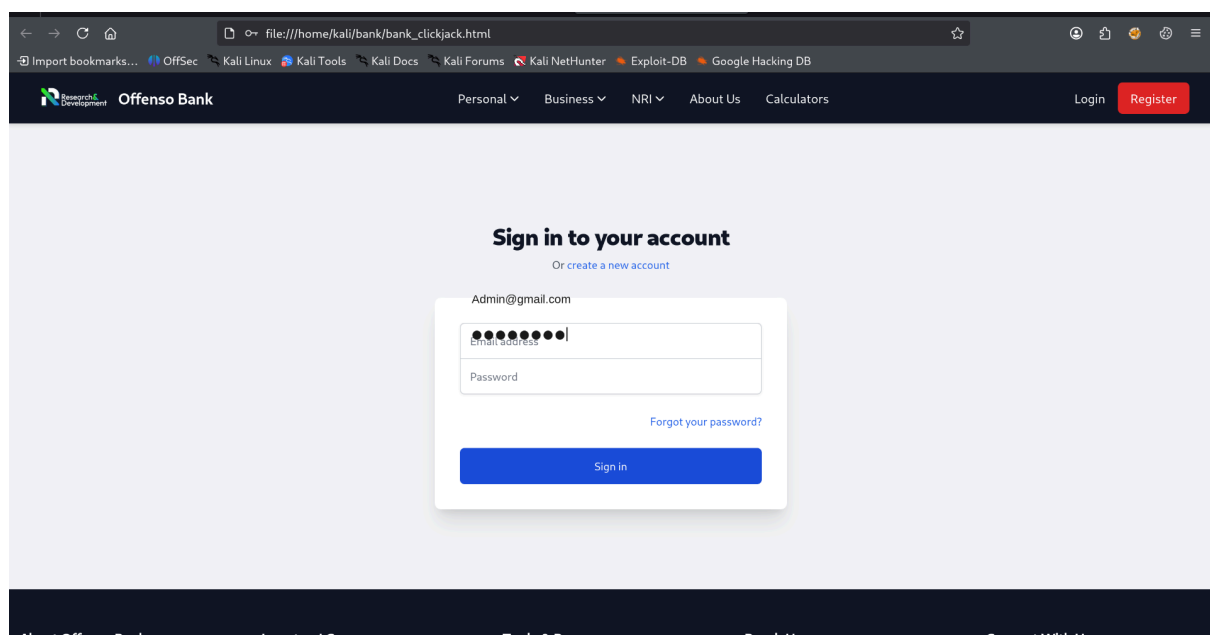
Proof of Concept

Step 1 – Verify the Login Page Can Be Framed

A custom HTML page was created that embeds the application's login page inside a full-screen iframe. The login page loaded successfully without any browser restrictions, confirming that framing by external websites is permitted.

Step 2 – Overlay Malicious Input Fields

Transparent input fields and a fake login button were positioned over the embedded login form to imitate the legitimate user interface. Users entering their credentials unknowingly interacted with the attacker's elements instead of the application's actual login form.



Step 3 – Capture User Credentials

As the victim entered their email address and password, the attacker-controlled server successfully received the supplied credentials, demonstrating that sensitive information could be stolen through the clickjacking attack.

```
[+] CAPTURED:  
  event : email  
  value : Admin@gmail.com
```

```
[+] CAPTURED:  
  event : password  
  value : Admin123
```

Impact

An attacker can host the banking application inside a malicious webpage and deceive authenticated or unauthenticated users into interacting with attacker-controlled elements. This can lead to credential theft, phishing attacks, unauthorized user actions, and potential account compromise.

Recommendation

Configure the X-Frame-Options response header as DENY or SAMEORIGIN. Implement Content-Security-Policy (CSP) using the frame-ancestors directive to restrict which origins are permitted to embed the application. Apply these protections to all sensitive pages, especially authentication, account management, and transaction-related pages.

F-07 – Sensitive Data Exposure Through Full Payment Card Number Disclosure

Severity	Medium
CVSS v3.1	5.5
Affected Endpoint	GET /#/dashboard

Description

The application displays the complete payment card number (Primary Account Number - PAN) on the user dashboard instead of masking sensitive digits. Exposing the full PAN unnecessarily increases the risk of sensitive information disclosure through shoulder surfing, screenshots, screen sharing, malware, or unauthorized observation. Payment card information should be masked unless full disclosure is explicitly required for a legitimate business purpose.

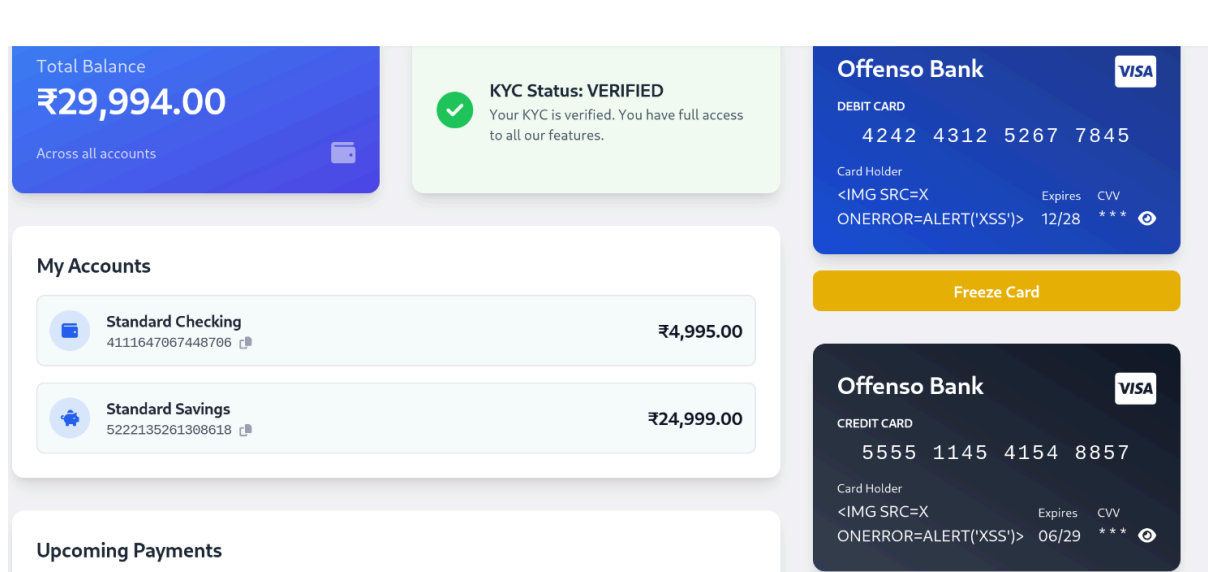
Proof of Concept

Step 1

Authenticate with a valid user account and navigate to the dashboard.

Step 2

Observe that both debit and credit cards display the complete 16-digit card number instead of masking the PAN.



Impact

An attacker or unauthorized individual with access to the user's screen, screenshots, or shared sessions may obtain the complete payment card number. This unnecessarily exposes sensitive cardholder information and increases the risk of information disclosure, social engineering, and potential payment fraud when combined with other compromised data.

Recommendation

Mask all payment card numbers by default (e.g., **** * 7845).
Display only the last four digits unless full disclosure is explicitly required.
Require additional user verification before revealing the complete card number.
Follow PCI DSS requirements for displaying and protecting cardholder data.

F-08 – Missing Rate Limiting on Authentication Endpoint

Severity	Medium
CVSS v3.1	5.3
Affected Endpoint	POST /api/auth/login

Description

The authentication endpoint does not implement effective rate limiting or account lockout mechanisms. Multiple consecutive login attempts can be performed without restriction, allowing an attacker to repeatedly submit password guesses against user accounts. Although Multi-Factor Authentication (MFA) is enforced after successful password validation, the absence of rate limiting increases the risk of password brute-force and credential stuffing attacks against the primary authentication factor.

Proof of Concept

Step 1

Intercept the login request and send it to Burp Suite Intruder.

Step 2

Launch a password brute-force attack using multiple password payloads.

Step 3

Observe that the application continues processing authentication requests without enforcing rate limiting, account lockout, or temporary blocking. A valid password returns a successful response and a `twoFactorToken`, while invalid passwords receive standard error responses.

Impact

An attacker can perform unrestricted password guessing or credential stuffing attacks against user accounts. While MFA provides an additional authentication layer, successful password guessing still allows the attacker to reach the second authentication stage, increasing the likelihood of account compromise if the second factor is bypassed, intercepted, or otherwise compromised.

Recommendation

Implement rate limiting on authentication requests.

Introduce temporary account lockout or exponential backoff after multiple failed login attempts.

Apply IP-based and account-based throttling.

Monitor and alert on excessive authentication failures.

Consider CAPTCHA after repeated failed login attempts.

F-09 – Business Logic Flaw Allowing Invalid Transfer Amounts

Severity	Medium
CVSS v3.1	5.4
Affected Endpoint	POST /api/transfer

Description

The transfer functionality relies on client-side validation to restrict invalid transfer amounts. However, by intercepting and modifying the request, the server accepts transfer amounts that should be rejected (e.g., `0.001`).

Although the account balance is not modified, the application returns a successful response and permanently records the transaction in the transaction history and exported reports. This indicates missing server-side business validation and results in inaccurate financial records.

Proof of Concept

Step 1

Log in to the application and initiate a normal transfer.

Step 2

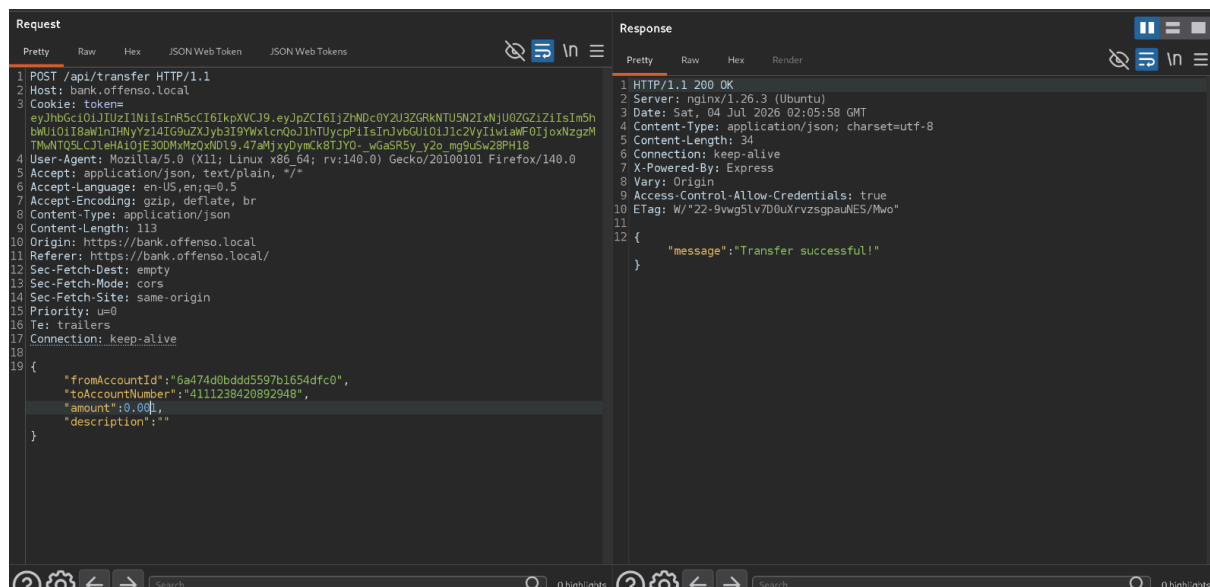
Intercept the request using Burp Suite and modify the transfer amount from the valid value to:

```
"amount": 0.001
```

Forward the modified request.

Step 3

Observe that the server accepts the request and returns:



despite the transfer amount being invalid.

Step 4

Navigate to **Transaction History** or **Export Transaction Report**.

The application records the transfer as a successful **TRANSFER_OUT** transaction with the modified amount (e.g., **-0.001**), even though the account balance remains unchanged.

To: 2948 - 1 7/3/2026, 5:17:32 PM	-₹1.00	Details
To: 2948 - hi 7/3/2026, 7:26:48 PM	-₹1.00	Details
To: 2948 - hi 7/3/2026, 7:27:04 PM	-₹1.00	Details
To: 2948 - hi 7/3/2026, 7:27:11 PM	-₹2.00	Details
To: 2948 - hi 7/3/2026, 7:27:34 PM	-₹0.00	Details
To: 2948 - Transfer 7/3/2026, 7:29:36 PM	-₹1.00	Details
To: 2948 - hi 7/3/2026, 7:32:35 PM	-₹0.00	Details
To: 2948 - hi	-₹0.00	Details

Impact

An attacker can create arbitrary low-value or invalid transfer records that are permanently stored in the application's transaction history and exported reports. This compromises the integrity of financial records and may mislead users, auditors, or administrative staff who rely on transaction logs. Repeated exploitation could pollute audit trails and reduce trust in financial reporting.

Recommendation

Enforce server-side validation for transfer amounts.

Reject values less than or equal to zero and any amount below the application's minimum supported transfer amount.

Validate business rules before creating or storing transaction records. Record transactions only after successful validation and processing. Return an appropriate error response (e.g., HTTP 400 Bad Request) when invalid amounts are submitted.

Conclusion

The Vulnerability Assessment and Penetration Test identified several security weaknesses affecting authentication, session management, client-side security, and business logic within the Offenso Bank web application. While no evidence of server compromise was observed during testing, the identified findings could increase the risk of unauthorized access, sensitive information disclosure, and application misuse if left unaddressed. It is recommended that the identified vulnerabilities be remediated according to their assigned severity and that regular security assessments be performed to maintain a strong security posture.

End of Report